Informe de laboratorio: expresiones regulares

Javier Siñuka

12 de septiembre de 2026

Índice

1. Resumen **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Marco teórico **3**

3.1. Concepto de expresión regular 3

3.2. Uso en ciberseguridad 4

3.3. Flujo conceptual de una búsqueda regex 4

4. Desarrollo de la práctica **4**

4.1. Paso 1: metacaracteres 4

4.2. Paso 2: interpretación de los patrones proporcionados 5

4.2.1. Consideraciones de precisión 6

4.3. Paso 3: verificación en CyberOps Workstation 6

4.3.1. Acceso y apertura del archivo 6

4.3.2. Matriz de verificación esperada 7

5. Análisis desde la perspectiva de ciberseguridad **7**

5.1. Lectura de logs y reducción del espacio de búsqueda 7

5.2. Falsos positivos y falsos negativos 7

5.3. Buenas prácticas 8

6. Resultados **8**

7. Conclusiones **8**

Referencia de la práctica **9**

1. Resumen

El presente informe resuelve integralmente la práctica de laboratorio dedicada al uso de expresiones regulares (*regular expressions*, regex). El trabajo aborda, en primer lugar, el significado de los principales metacaracteres utilizados para construir patrones; posteriormente interpreta cada una de las expresiones regulares propuestas en la guía; y, finalmente, documenta el procedimiento de validación sobre el archivo de registro `logstash-tutorial.log` mediante la herramienta `less` en la máquina virtual CyberOps Workstation.

Desde la perspectiva de ciberseguridad, las expresiones regulares constituyen una herramienta fundamental para el análisis de logs, la búsqueda de indicadores de compromiso, la identificación de patrones de tráfico y la extracción de campos relevantes en procesos de monitoreo, respuesta a incidentes y análisis forense. Su uso debe realizarse con precisión, ya que un patrón excesivamente amplio puede producir falsos positivos, mientras que uno demasiado restrictivo puede omitir eventos relevantes.

2. Objetivos

2.1. Objetivo general

Comprender y aplicar expresiones regulares para identificar cadenas de información dentro de archivos de texto y registros de eventos, interpretando correctamente sus operadores y verificando su comportamiento en un entorno de análisis CyberOps.

2.2. Objetivos específicos

- Reconocer la función de los principales metacaracteres de una expresión regular.
- Interpretar patrones regex simples y compuestos.
- Aplicar anclas, clases de caracteres, cuantificadores, escapes y alternancias.
- Ejecutar búsquedas dentro de un archivo de log utilizando `less`.
- Relacionar el uso de expresiones regulares con tareas habituales de ciberseguridad.

3. Marco teórico

3.1. Concepto de expresión regular

Una expresión regular es una notación formal para describir conjuntos de cadenas de caracteres mediante patrones. A diferencia de una búsqueda literal, una regex puede representar familias completas de cadenas. Por ejemplo, el patrón `[0-9]` representa cualquier dígito decimal, mientras que `[A-Z]{2,4}` representa una secuencia de entre dos y cuatro letras mayúsculas.

La guía de laboratorio menciona dos grandes familias históricas de sintaxis: POSIX y Perl. En la práctica moderna existen múltiples motores compatibles en distinto grado con esas convenciones. Por este motivo, ciertos detalles —por ejemplo, el significado exacto de `\d` o el comportamiento de `.` frente a saltos de línea— pueden variar según la herramienta utilizada. Para este informe se adopta la interpretación esperada por la práctica de Cisco y por motores de expresiones regulares de uso habitual.

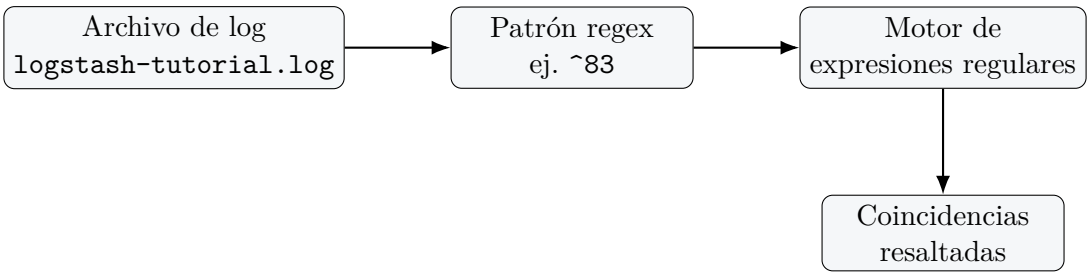
3.2. Uso en ciberseguridad

En un centro de operaciones de seguridad (SOC), las regex se emplean habitualmente para:

- localizar direcciones IP, dominios, nombres de host, identificadores o marcas temporales;
- filtrar eventos de autenticación o solicitudes HTTP específicas;
- construir reglas de detección y extracción de campos;
- identificar cadenas asociadas a indicadores de compromiso (IoC);
- normalizar datos provenientes de fuentes de log heterogéneas;
- realizar búsquedas rápidas durante una investigación forense.

Debe considerarse, además, que patrones mal diseñados pueden ocasionar problemas de rendimiento. En motores susceptibles a retroceso excesivo, determinadas expresiones pueden provocar *Regular Expression Denial of Service* (ReDoS). Por ello, en sistemas de producción resulta importante mantener patrones acotados, legibles y probados sobre entradas representativas.

3.3. Flujo conceptual de una búsqueda regex



4. Desarrollo de la práctica

4.1. Paso 1: metacaracteres

La primera actividad solicita describir la función de los metacaracteres indicados. La tabla siguiente completa la totalidad de los elementos presentados en la guía.

Metacarácter	Descripción
`$`	Ancla de fin. Hace coincidir una posición situada al final de la cadena o de la línea, según el modo del motor regex. No consume caracteres.
`*`	Cuantificador de repetición: el elemento inmediatamente anterior puede aparecer cero o más veces. Por ejemplo, `ab*` coincide con `a`, `ab`, `abb`, etc.
`.`	Comodín que representa un carácter cualquiera, salvo normalmente el salto de línea, dependiendo de la configuración del motor.

Metacarácter	Descripción
`[ ]`	Define una clase de caracteres. Coincide con exactamente un carácter perteneciente al conjunto o rango indicado entre corchetes. Ejemplo: `[A-Z]`.
`\.`	Representa un punto literal. La barra invertida elimina el significado especial del metacarácter `.`.
`\d`	Coincide con un dígito. En la interpretación habitual del ejercicio equivale a un carácter de `0` a `9`.
`\D`	Coincide con un carácter que no es un dígito; es la clase complementaria de `\d`.
`^`	Ancla de inicio. Exige que la coincidencia comience al principio de la cadena o línea, según el modo de búsqueda. No consume caracteres.
`{m}`	Cuantificador exacto: el elemento anterior debe repetirse exactamente m veces.
`{n,m}`	Cuantificador de rango: el elemento anterior debe repetirse como mínimo n veces y como máximo m veces, inclusive.
`$`	La guía repite este metacarácter. Su función es la misma: anclar la coincidencia al final de la cadena o línea.
`*`	La guía repite este metacarácter. Su función es la misma: permitir cero o más repeticiones del elemento anterior.
`abc|123`	Alternancia lógica. Coincide con una de las dos alternativas: la cadena literal `abc` o la cadena literal `123`.

Observación: la tabla original presenta los metacaracteres `$` y `*` dos veces. Para conservar fielmente la estructura de la práctica, ambos se mantienen y se responden nuevamente.

4.2. Paso 2: interpretación de los patrones proporcionados

A continuación se resuelve cada expresión regular solicitada por la guía.

Patrón	Interpretación correcta
`^83`	Coincide únicamente cuando la cadena o línea comienza con los caracteres literales `83`. El símbolo `^` fija el inicio de la búsqueda.
`[A-Z]{2,4}`	Coincide con una secuencia de entre dos y cuatro letras mayúsculas ASCII consecutivas. El rango `[A-Z]` define las letras permitidas y `{2,4}` establece la cantidad.
`2015`	Busca la secuencia literal `2015` en cualquier posición del texto. No contiene metacaracteres.
`05:22:2[0-9]`	Coincide con una hora o fragmento de marca temporal que empieza con `05:22:2` y termina con cualquier dígito. Por lo tanto, puede coincidir con `05:22:20`, `05:22:21`, ..., `05:22:29`.
`\.com`	Coincide con la cadena literal `.com`. La secuencia `\.` obliga a interpretar el punto como carácter literal y no como comodín.

Patrón	Interpretación correcta
`complete|GET`	Alternancia: coincide con la cadena literal `complete` o con la cadena literal `GET`.
`0{4}`	Coincide con exactamente cuatro caracteres `0` consecutivos, es decir, `0000`.

4.2.1. Consideraciones de precisión

Es importante destacar que los patrones anteriores no incorporan límites de palabra ni anclas adicionales salvo cuando se indican explícitamente. En consecuencia:

- `2015` también coincidiría dentro de una cadena más extensa que contenga esos cuatro caracteres.
- `0{4}` puede encontrar cuatro ceros dentro de una secuencia más larga de ceros.
- `[A-Z]{2,4}` puede identificar fragmentos de dos a cuatro mayúsculas dentro de una secuencia mayor, dependiendo del motor y de su estrategia de coincidencia.

Estas observaciones son relevantes en ciberseguridad: una regla de detección debe expresar exactamente el contexto que se desea identificar para reducir falsos positivos.

4.3. Paso 3: verificación en CyberOps Workstation

La guía indica verificar los patrones sobre el archivo `logstash-tutorial.log` almacenado en la máquina virtual CyberOps Workstation. El procedimiento completo es el siguiente.

4.3.1. Acceso y apertura del archivo

Iniciar sesión en la VM con las credenciales proporcionadas por la práctica y abrir una terminal. A continuación:

```
cd lab.support.files/
less logstash-tutorial.log
```

Dentro de `less`, una búsqueda se inicia escribiendo `/` seguida de la expresión regular. Las expresiones solicitadas deben probarse individualmente de la siguiente manera:

```
/^83
/[A-Z]{2,4}
/2015
/05:22:2[0-9]
/\.com
/complete|GET
/0{4}
```

Una vez ingresado cada patrón, `less` resalta la coincidencia encontrada. Se puede navegar por el archivo mediante las teclas `j` y `k`; adicionalmente, en una sesión habitual de `less`, `n` permite avanzar a la coincidencia siguiente y `N` regresar a la anterior. Finalmente, se presiona `q` para salir.

4.3.2. Matriz de verificación esperada

Patrón	Qué debe verificarse	Criterio de aceptación
`^83`	Inicio de línea/cadena con `83`.	Solo se resaltan coincidencias que comienzan en la primera posición.
`[A-Z]{2,4}`	Bloques de 2 a 4 letras mayúsculas.	Deben encontrarse secuencias como métodos, siglas o fragmentos en mayúsculas.
`2015`	Cadena literal `2015`.	Se resaltan todas las apariciones del año/secuencia.
`05:22:2[0-9]`	Tiempos entre `05:22:20` y `05:22:29`.	El último carácter debe ser un dígito entre 0 y 9.
`\.com`	Sufijo literal `.com`.	El punto debe formar parte de la coincidencia.
`complete|GET`	Alternativas `complete` o `GET`.	Deben resaltarse apariciones de cualquiera de ambas cadenas.
`0{4}`	Cuatro ceros consecutivos.	La coincidencia mínima visible es `0000`.

Nota metodológica: la guía entregada contiene las instrucciones y una captura ilustrativa, pero no incorpora como archivo adjunto el contenido completo de `logstash-tutorial.log`. Por ello, este informe no inventa una lista de líneas coincidentes. La resolución incluye la interpretación exacta de todos los patrones y el procedimiento reproducible para comprobarlos en la VM indicada.

5. Análisis desde la perspectiva de ciberseguridad

5.1. Lectura de logs y reducción del espacio de búsqueda

El análisis manual de un archivo de registro extenso puede resultar costoso. Las expresiones regulares permiten reducir rápidamente el conjunto de eventos candidatos. Por ejemplo, la búsqueda `complete|GET` puede localizar eventos que contengan un estado o una operación HTTP específica, mientras que `\.com` puede ayudar a identificar dominios de nivel superior `.com` dentro de URLs o nombres de host.

No obstante, una coincidencia textual no constituye por sí sola evidencia de actividad maliciosa. En un proceso profesional de monitoreo, el analista debe correlacionar el resultado con otros campos: dirección IP, usuario, código de estado, marca temporal, agente de usuario, host afectado y contexto del evento.

5.2. Falsos positivos y falsos negativos

Un patrón demasiado general, como `2015`, puede generar numerosos resultados irrelevantes. Por el contrario, un patrón demasiado específico puede omitir variantes legítimas del dato buscado. El diseño de reglas de detección requiere equilibrar cobertura y precisión.

Como ejemplo, si el objetivo real fuera detectar exclusivamente el año `2015` como token indepen-

diente, una expresión más estricta podría incorporar límites de palabra, por ejemplo `\b2015\b`, siempre que el motor empleado soporte dicha construcción. Esto ilustra la diferencia entre resolver el ejercicio —donde debe utilizarse exactamente el patrón indicado— y diseñar una regla de producción.

5.3. Buenas prácticas

- Documentar la intención de cada patrón.
- Evitar complejidad innecesaria y retroceso catastrófico.
- Probar patrones con casos positivos y negativos.
- Utilizar anclas y límites cuando el contexto lo requiera.
- Escapar correctamente caracteres con significado especial.
- Validar el comportamiento en el motor regex real del sistema objetivo.
- No utilizar regex como único mecanismo de validación de seguridad cuando se requiere un parser estructurado.

6. Resultados

La práctica permite comprobar los siguientes resultados:

1. Se identificó correctamente la función de todos los metacaracteres solicitados.
2. Se interpretaron sin ambigüedad los siete patrones del Paso 2.
3. Se documentó el procedimiento completo para buscar cada patrón dentro de `logstash-tutorial.log` utilizando `less`.
4. Se vinculó el contenido del laboratorio con aplicaciones reales de análisis de registros y detección en ciberseguridad.

7. Conclusiones

Las expresiones regulares proporcionan un mecanismo compacto y potente para describir patrones de texto. En el contexto de CyberOps, su utilidad radica en permitir búsquedas rápidas y repetibles dentro de grandes volúmenes de registros. La práctica analizada introduce correctamente conceptos esenciales: anclas, cuantificadores, clases de caracteres, escapes y alternancias.

La correcta interpretación de operadores como `^`, `$`, `*`, `[ ]`, `{m}` y `|` es indispensable para evitar búsquedas imprecisas. Asimismo, el ejercicio de verificación mediante `less` muestra cómo una regex puede utilizarse directamente en una herramienta de línea de comandos para localizar eventos relevantes.

Desde una perspectiva profesional, el principal aprendizaje es que una expresión regular debe evaluarse no solo por su capacidad de producir coincidencias, sino también por su exactitud semántica, su eficiencia y su adecuación al motor que la ejecuta. Estos criterios son especialmente importantes cuando las regex forman parte de reglas de monitoreo, sistemas SIEM, pipelines de procesamiento de logs o mecanismos automáticos de detección.

Referencia de la práctica

Cisco Networking Academy. *Práctica de laboratorio: Tutorial de expresiones regulares*. Material de laboratorio CyberOps proporcionado para la actividad.